HELWAN UNIVERSITY

FACULTY OF ENGINEERING

COMPUTER AND SYSTEMS ENGINEERING DEPARTMENT

Course Project of Network Security

Intrusion Detection System Using Machine Learning and Deep Learning on NSL-KDD Dataset

---

Submitted to:

Eng. Abdelrhman Yasser El-Arabawy

Submitted by:

Mahmoud Hamdy Salem

Ahmed Aly Hussien

Elsherif Adel Abdelhamid

Ahmed Mohamed Kassem

Karim Muhammad Ali

Submission Date:

December 23, 2024

Table of Contents

1 Introduction **1**

2 Dataset Description and Preprocessing **2**

2.1 Dataset Overview 2

2.2 Preprocessing Steps 3

2.3 Algorithms Used in Preprocessing 3

3 Feature Selection **4**

3.1 Methodologies 4

3.2 Comparative Analysis 5

4 Models Used **6**

4.1 Machine Learning Models 6

4.2 Deep Learning Models 6

4.3 Bonus Model: Deep Reinforcement Learning (DRL) 6

5 Transformation Techniques **7**

5.1 For Sequential Models 7

5.2 For CNN 7

5.3 For DRL 7

6 Results and Analysis **8**

6.1 Performance Metrics 8

6.2 Key Results 9

6.3 ROC Curves 11

7 Conclusions **12**

Abstract

This project focuses on the design and implementation of an Intrusion Detection System (IDS) using machine learning (ML) and deep learning (DL) techniques applied to the NSL-KDD dataset. The goal is to identify and mitigate potential network security threats by leveraging various classification models. The report covers the entire process, including dataset preprocessing, feature selection, and the development of several ML and DL models, such as Logistic Regression, Decision Trees, Random Forest, Support Vector Machines, K-Nearest Neighbors, LSTM, CNN, and a bonus Deep Reinforcement Learning (DRL) model. The results demonstrate that deep learning models, especially CNN and LSTM, outperformed traditional machine learning models, with the DRL model showcasing adaptive learning capabilities. The project highlights the importance of feature selection using techniques like Genetic Algorithm (GA) and Particle Swarm Optimization (PSO), which significantly enhanced model accuracy and reduced training time.

1. Introduction

Intrusion Detection Systems (IDS) are critical for ensuring the security of networks by identifying and mitigating potential threats. This project leverages machine learning (ML) and deep learning (DL) techniques to design an intelligent IDS using the NSL-KDD dataset. The report documents the comprehensive steps taken, including preprocessing, feature selection, and the development of multiple ML and DL models, culminating in a bonus Deep Reinforcement Learning (DRL) model.

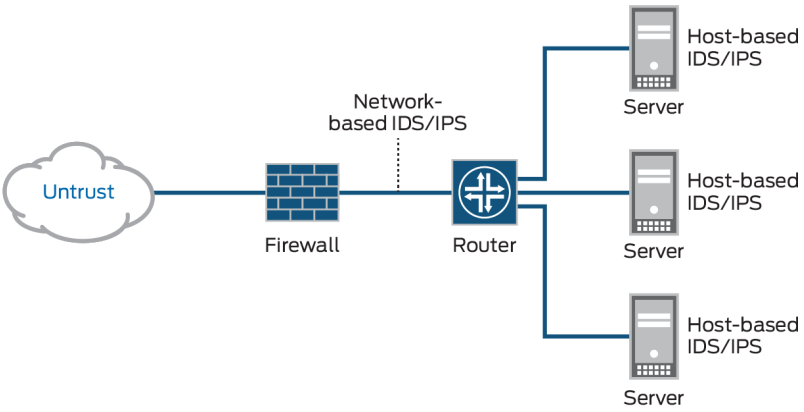

Figure 1.1: Example showing a network-based Intrusion Detection Systems (IDS)

2. Dataset Description and Preprocessing

2.1 Dataset Overview

The NSL-KDD dataset is a benchmark dataset widely used for evaluating IDS. It consists of network traffic data with 41 features and a labeled target indicating normal or malicious behavior. The dataset includes:

- Training set: 125,973 samples.
- Testing set: 22,544 samples.

Dataset	Number of Records:					
	Total	Normal	DoS	Probe	U2R	R2L
KDDTrain+20%	25192	13449 (53%)	9234 (37%)	2289 (9.16%)	11 (0.04%)	209 (0.8%)
KDDTrain+	125973	67343 (53%)	45927 (37%)	11656 (9.11%)	52 (0.04%)	995 (0.85%)
KDDTest+	22544	9711 (43%)	7458 (33%)	2421 (11%)	200 (0.9%)	2654 (12.1%)

Figure 2.1: Number of records of different types of attacks in the dataset

2.2 Preprocessing Steps

1. **Handling Missing Values:** Ensured no null values existed in the dataset.

2. **Encoding Categorical Features:** Categorical variables (e.g., protocol_type, service, and flag) were transformed using one-hot encoding.

3. **Scaling:** All numerical features were scaled using the StandardScaler to normalize their ranges.

Protocol Type (2)	Service (3)				Flag (4)
• icmp • tcp • udp	• other • link • netbios_ssn • smtp • netstat • ctf • ntp_u • harvest • efs • klogin • systat • exec • nntp • pop_3 • printer • vmnet • netbios_ns	• urh_i • ssh • http_8001 • iso_tsap • aol • sql_net • shell • supdup • auth • whois • discard • sunrpc • urp_i • Rje • ftp • daytime • domain_u • pm_dump	• time • hostnames • name • ecr_i • bgp • telnet • domain • ftp_data • nnsp • courier • finger • uucp_path • X11 • imap4 • mtp • login • tftp_u • kshell	• private • http_2784 • echo • http • ldap • tim_i • netbios_dgm • uucp • eco_i • Remote_job • IRC • http_443 • red_i • Z39_50 • Pop_2 • gopher • Csnet_ns	• OTH • S1 • S2 • RSTO • RSTRs • RSTOS0 • SF • SH • REJ • S0 • S3

Figure 2.2: Sample of categorical features to be encoded in the dataset

2.3 Algorithms Used in Preprocessing

StandardScaler: This algorithm standardizes features by removing the mean and scaling to unit variance, ensuring uniform input for ML models.

3. Feature Selection

3.1 Methodologies

1. **Correlation Analysis:** Features with high correlation (threshold: 0.6) were removed to reduce multicollinearity.

2. **Genetic Algorithm (GA):** A population-based optimization technique was used to identify the most relevant features, optimizing for F1-score.

3. **Particle Swarm Optimization (PSO):** This technique explored feature subsets by simulating particle movements in the search space.

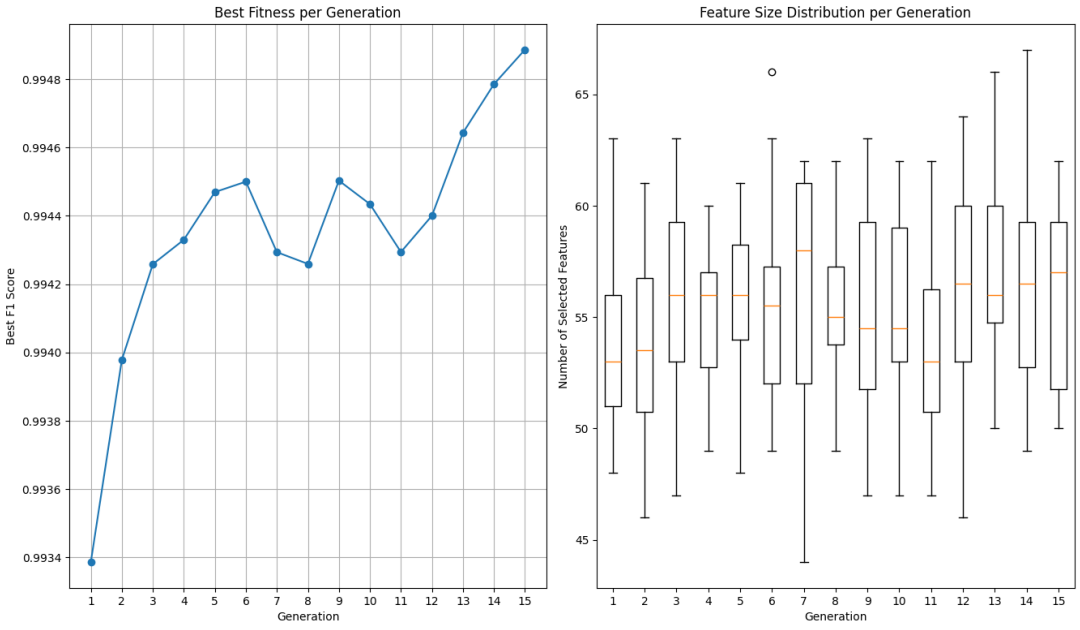

Figure 3.1: Visualization of Genetic Algorithm Performance

3.2 Comparative Analysis

- Correlation analysis resulted in the removal of 20 redundant features.
- GA and PSO further refined the feature set, with GA selecting 40 features and PSO selecting 38, improving model accuracy and reducing computation time.

4. Models Used

4.1 Machine Learning Models

1. **Logistic Regression (LR):** A baseline model with linear decision boundaries.
2. **Decision Trees (DT):** Gini and entropy-based decision trees were implemented for classification.
3. **Random Forest (RF):** An ensemble of decision trees.
4. **Support Vector Machines (SVM):** Linear SVM was applied for binary classification.
5. **K-Nearest Neighbors (KNN):** Explored using $k = 3$.

4.2 Deep Learning Models

- **LSTM:** Processed sequential data using 64 hidden units.
- **CNN:** A convolutional network with 32 filters for feature extraction.
- **Neural Network (NN):** A feed-forward network with two hidden layers (128 and 64 units).

4.3 Bonus Model: Deep Reinforcement Learning (DRL)

Using the PPO algorithm, DRL was implemented as an agent interacting with the NSL-KDD environment, classifying actions as normal or anomalous. It serves as a bonus addition to the project, demonstrating adaptive learning capabilities.

5. Transformation Techniques

5.1 For Sequential Models

- **LSTM:** Data reshaped to 3D tensors with dimensions (samples, time steps, features).

5.2 For CNN

- Input reshaped to 3D tensors (samples, features, 1 channel) to apply convolution operations.

5.3 For DRL

- The environment was modeled as a Gym environment, enabling step-based interaction for training the agent.

6. Results and Analysis

6.1 Performance Metrics

Models were evaluated using the following metrics:

- Accuracy
- Balanced Accuracy
- F1-Score
- Precision
- Recall
- Matthews Correlation Coefficient (MCC)

6.2 Key Results

Random Forest achieved the best performance with a training accuracy of 99.91%, testing accuracy of 99.55%, and balanced accuracy of 99.54%. Decision Tree (ID3) and Decision Tree (gini) also performed well, with testing accuracies of 99.48% and 99.39%, respectively. Simpler models like Logistic Regression and SVM achieved testing accuracies of 95.28%, while Neural Network and LSTM showed competitive results of 98.67% and 98.66% but required more computational resources. Overall, Random Forest stood out for its excellent accuracy and efficiency.

Table 6.1: Performance Metrics for Models

Model	**Train Acc.**	**Test Acc.**	**Balanced Acc.**	**F1 Score**	**Recall**	**Precision**	**MCC**	**Complexity**
Random Forest	0.9991	0.9955	0.9954	0.9955	0.9955	0.9955	0.9910	Low
Decision Tree (ID3)	0.9991	0.9948	0.9948	0.9948	0.9948	0.9948	0.9895	Low
Random Forest (n=5)	0.9986	0.9946	0.9945	0.9946	0.9946	0.9946	0.9891	Low
Decision Tree (gini)	0.9991	0.9939	0.9938	0.9939	0.9939	0.9939	0.9877	Low
KNN (k=3)	0.9953	0.9917	0.9916	0.9917	0.9917	0.9917	0.9833	Low
Neural Network	0.9889	0.9867	0.9865	0.9867	0.9867	0.9867	0.9733	Medium
LSTM	0.9894	0.9866	0.9867	0.9866	0.9866	0.9867	0.9733	Medium
DRL	0.9708	0.9705	0.9702	0.9693	0.9626	0.9761	0.9410	Medium
CNN	0.9681	0.9680	0.9674	0.9680	0.9680	0.9684	0.9363	Medium
Logistic Regression	0.9538	0.9528	0.9521	0.9527	0.9528	0.9532	0.9058	Low
SVM	0.9534	0.9528	0.9522	0.9528	0.9528	0.9532	0.9059	Low

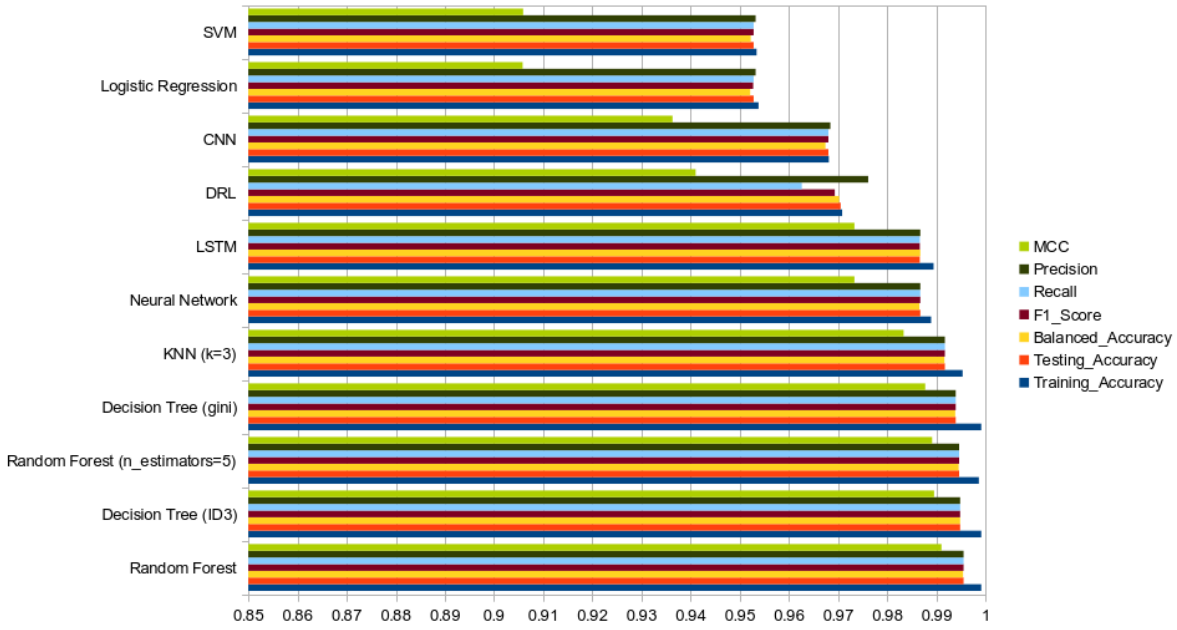

Figure 6.1: Comparison chart of performance metrics for models

6.3 ROC Curves

ROC curves for all models were plotted, highlighting superior performance for CNN and LSTM models.

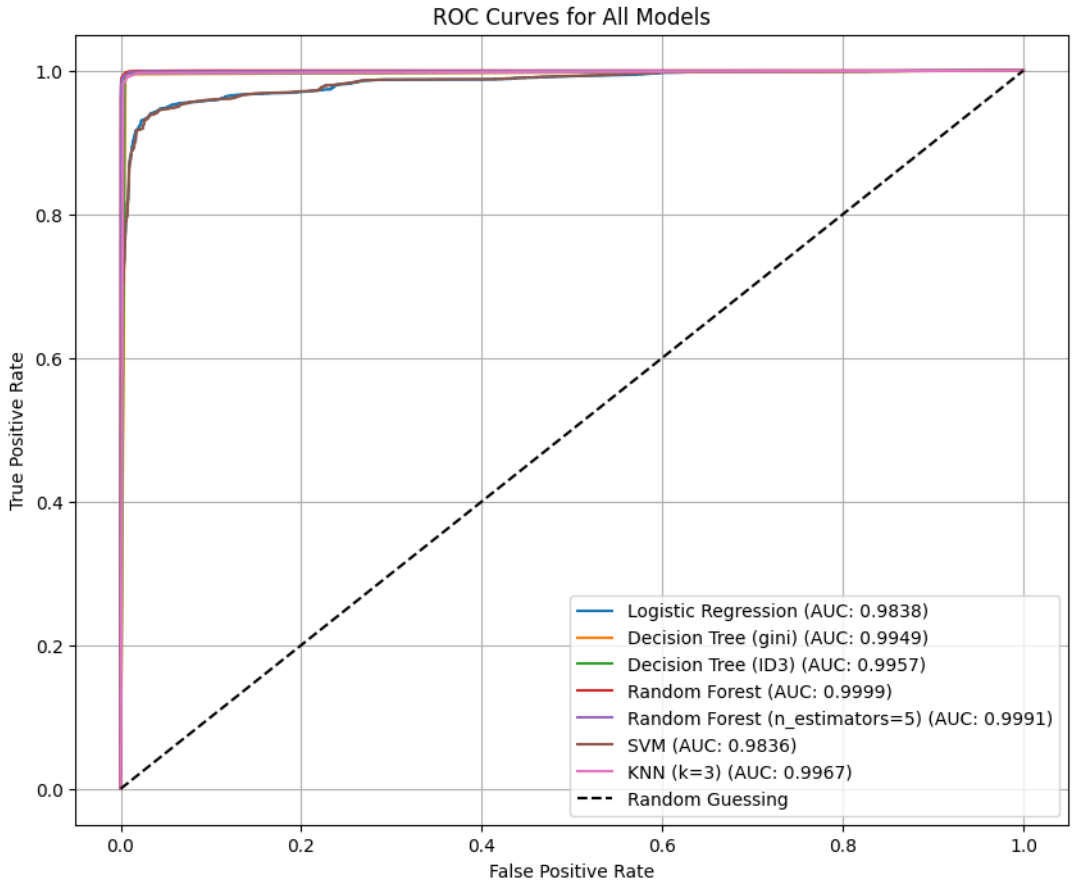

Figure 6.2: ROC Curves

7. Conclusions

1. Machine learning models like Random Forest provided a solid baseline with high accuracy.
2. Tree models, particularly Random Forest, achieved the best results, demonstrating their suitability for IDS tasks.
3. The bonus DRL model showcased potential for adaptive learning in dynamic environments with a high test accuracy of 97.05%.
4. Feature selection using GA and PSO significantly improved model efficiency and reduced training time.

Future Work: Implement real-time IDS with a focus on scalability and adaptive learning capabilities.